

Lab Assignment 5: VERILOG PARAMETERS

Module: Verilog & Logic Synthesis

Overview:

This assignment evaluates advanced Verilog concepts, including preprocessor macro scoping, parameter derivation, and specify block timing. Students will apply these theoretical principles practically by designing and verifying a parameterized, timing-aware Cryptographic ALU.

Part 1: Theoretical & Analytical Assignment

Q1: The Scope of the Preprocessor (`define)

Module A and Module B are compiled in the same compilation unit in the following order:

```
// File 1
`define DATA_WIDTH 16
module A (input [`DATA_WIDTH-1:0] in, output [`DATA_WIDTH-1:0] out);
    assign out = in;
endmodule
```

```
// File 2
module B (input [`DATA_WIDTH-1:0] in, output [`DATA_WIDTH-1:0] out);
    assign out = in;
endmodule
```

- Will Module B compile successfully? Why or why not? Explain how the preprocessor handles macro definitions across file boundaries.
- If DATA_WIDTH needs to be restricted only to Module A, how should the code be modified without resorting to module parameters?
- Given that `define is a text substitution macro, at which exact phase are macros resolved compared to parameter overrides? If a module uses a macro to define the default value of a parameter, explain how the compiler and elaborator process this sequence.

Q2: Parameter vs. Localparam Derivation

Consider the following module:

```
module tricky_math #(parameter BASE = 8) (input clk);
    parameter MULT = BASE * 2;
    localparam DIV = MULT / 2;
    // ...
endmodule
```

```
module tb_tricky_math;
    reg clk;
    tricky_math #(.BASE(4), .MULT(16), .DIV(2)) uut (.clk(clk));
    initial clk = 0;
    always #5 clk = ~clk;
endmodule
```

- Which of the overrides will cause a compilation/elaboration error and why?
- What will be the final calculated value of DIV if instantiated as **tricky_math #(.BASE(10),.MULT(4)) uut (.clk(clk));** ?

Q3: The Override Clash: Inline vs. Hierarchical

Look at the following code.

```
module top;
  // Inline parameter override
  fifo #(.WIDTH(8)) my_fifo (
    .clk(clk),
    .data(data)
  );

  // Hierarchical defparam override
  defparam top.u_fifo.WIDTH = 16;
endmodule
```

- What will be the final value of the parameter WIDTH inside my_fifo when synthesized? And why?

Q4: The Multiple Driver Race Condition

In a large SoC design,

```
// File A contains:
defparam top.cpu.alu.DATA_WIDTH = 32;
```

```
// File B contains:
defparam top.cpu.alu.DATA_WIDTH = 64;
```

- Explain what will happen during compilation and elaboration?

Q5: Timing with specparam

- Can a specparam be declared outside of a specify...endspecify block?
- Can a standard parameter be used to define a delay inside a specify block?
- Explain how specparam values are typically overridden in post-synthesis gate-level simulations. (Hint: Detail the role of Standard Delay Format (SDF) files).

Q6: The Simulation Flow

- Draw the standard simulation flow diagram.
- Briefly contrast the purpose of **Compilation** vs **Elaboration** vs **Simulation**. Provide the standard command used for each phase.

Q7: Error Isolation

During the execution of a design, a developer connects a 16-bit output port to an 8-bit input port on a sub-module.

- According to the standard simulation flow diagram, during which specific phase will this connection consistency error (mismatched signal widths) be caught?
- Which tool/command is responsible for performing this specific phase?

Part 2: Lab - Parameterized Memory Design

1. RTL Design Requirements

• Macro Definition:

Define a text substitution macro named POW2(n) that calculates 2^n .

• Module Interface (Parameters):

Use an ANSI-C style port list. Declare two Parameters: **ADDR_WIDTH (default: 10)** and **DATA_WIDTH (default: 32)**.

• Internal Constants (Localparam):

Calculate a constant named **MEM_DEPTH** internally using your **POW2** macro and **ADDR_WIDTH**. Use it to size the memory array.

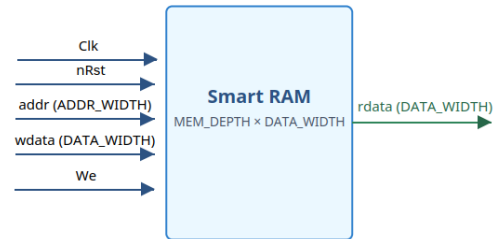
- **Asynchronous negative edge reset:** On the **negative edge of the reset**, reset all memory elements to 0.

- **Synchronous write:** On the **positive edge of the clock**, if **We** is high, write the **wdata** into the memory element specified by **addr**.

- **synchronous read:** On the **positive edge of the clock**, if **We** is Low, output the value of the memory element specified by **addr** to **rdata**.

• Timing Constants (Specparam):

Create a specify block declaring **t_setup = 2.0** and **t_hold = 1.0**. Implement timing checks for **wdata** and **We** against **clk**.



2. The Simulation Flow Execution

Using a testbench that overrides **ADDR_WIDTH to 8** and **DATA_WIDTH to 16**, **execute** and **answer**:

- **Compilation:** Run **vlog**. Did the parser identify any syntax or semantic errors?
- **Elaboration:** Run **vsim**. Did the tool successfully link the design units and resolve the overridden parameters?
- **Simulation:** Use **run**. Does the functional behavior match your specified requirements in the waveform viewer?

NOTE:

INITIALIZE MEMORY IN VERILOG:

- <https://projectcf.io/posts/initialize-memory-in-verilog/>

Deliverables

Submit your **report** containing **all the answers** and your **.v files** containing all requested modules, alongside a **brief testbench (tb.v)** simulating the modules to verify correct operation.